

Image Enhancement in the Spatial Domain

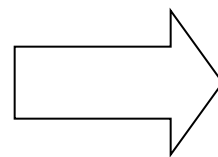
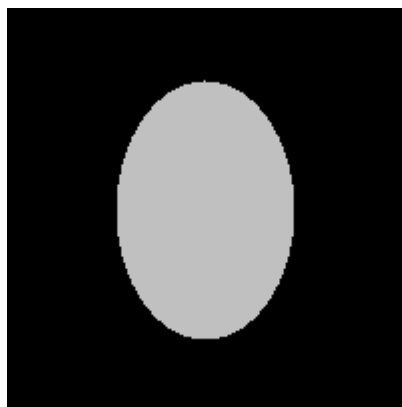
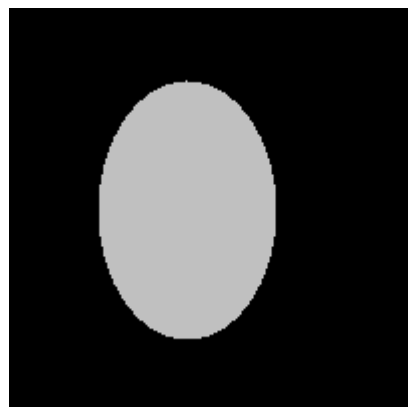
Spatial Filtering

Image Operations

- Arithmetic Operations on pixels (p and q are images/image matrices)
 - $p + q$
 - $p - q$
 - $p * q$ (also stated as pq , or $p \times q$)
 - $p \% q$
- Logic Operations on pixels (p and q are binary images/image matrices)
 - $p \text{ AND } q$
 - $p \text{ OR } q$
 - $\text{Not } q$
 - $p \text{ XOR } q$

Image Operations

- Subtraction or Absolute Difference
 - Very useful for determining the difference between two images
 - Used for “detection” of changes

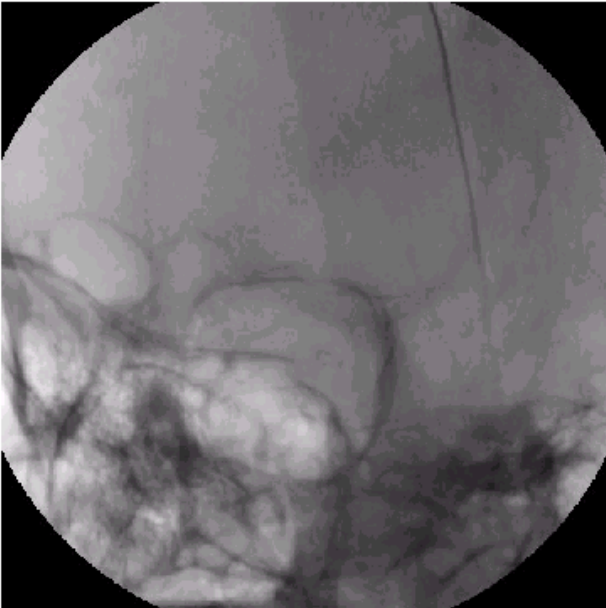


Absolute Difference

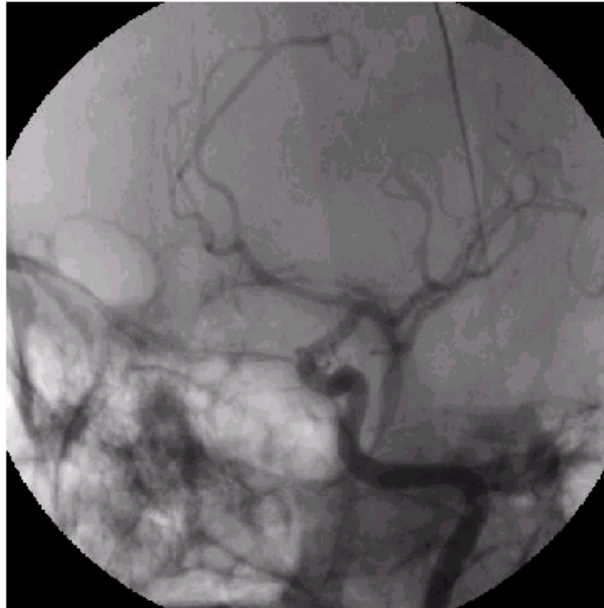


Grey = non-zero, Background = 0.

Example



X-ray brain image
without contrast agent
injected



X-ray brain image
with contrast agent injected

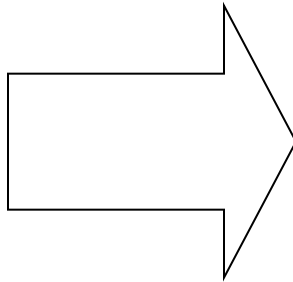
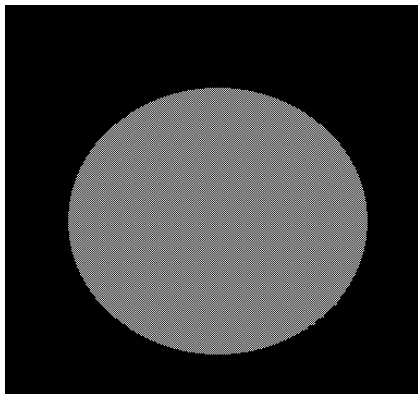
Digital subtraction angiogram
(DSA)



http://en.wikipedia.org/wiki/Digital_subtraction_angiography

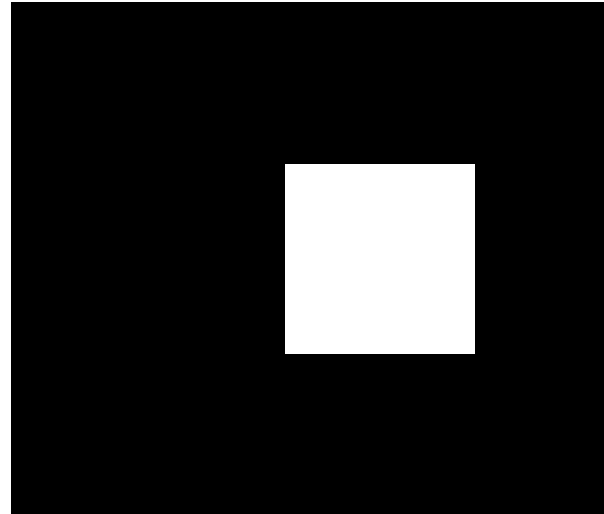
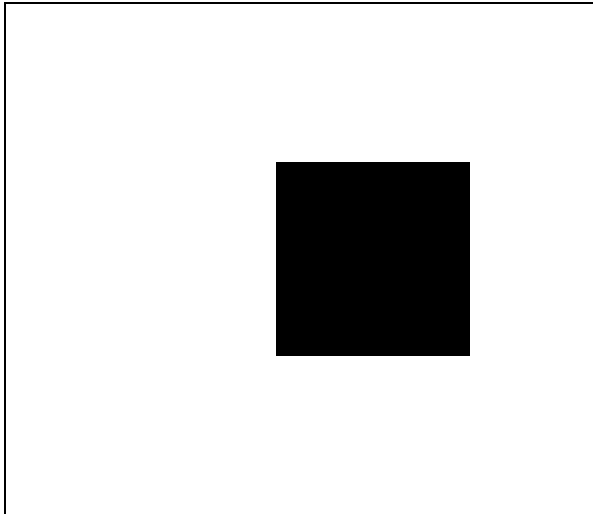
Image Operation

- Addition can blend two images



Logic Operators

- NOT

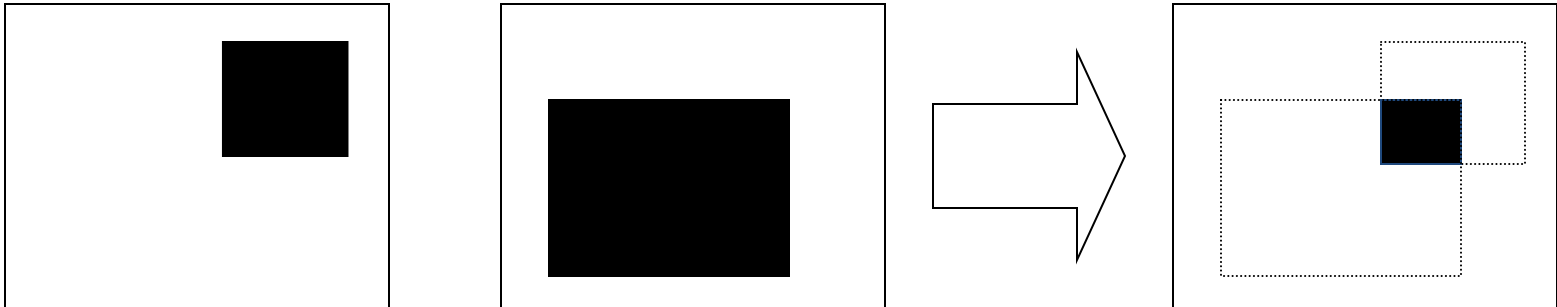


black = 1, white = 0.

Logic Operators

- AND

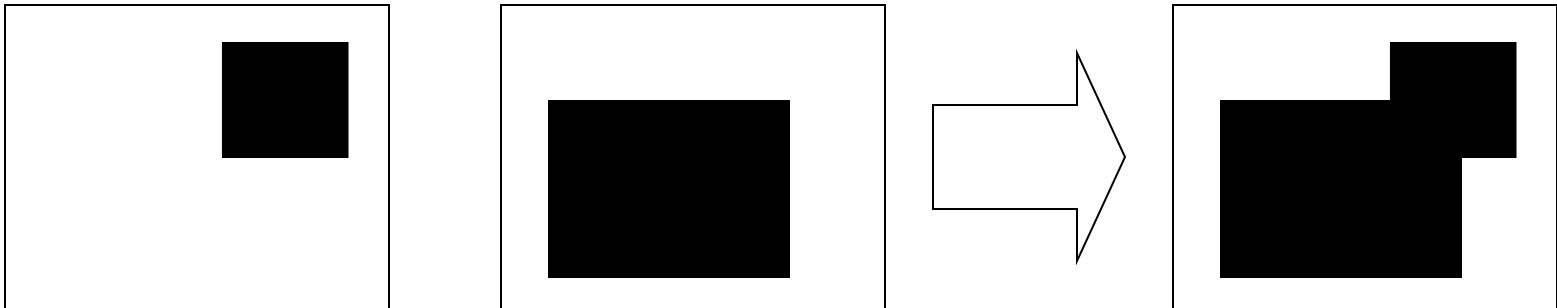
black = 1, white = 0.



Logic Operators

- OR

black = 1, white = 0.



- XOR?

Image Averaging

- If you can capture a scene or acquire an image repeatedly, then averaging all the observed images will give you a clear image with less noise.
- $g_i(x,y)$ = i^{th} observed noisy image, $i = 1, \dots, K$, and K = total number of images.
- $f(x,y)$ = 'true', original image
- $\eta(x,y)$ = (*eta*) is zero mean, random Gaussian noise
- the final average image is given by

$$\bar{g}(x, y) = \frac{1}{K} \sum_{i=1}^K g_i(x, y)$$

Image Averaging

- It follows that

Expected value of the
average image

$$E \{ \bar{g}(x, y) \} = f(x, y)$$

Variance of the
average image

$$\sigma_{\bar{g}(x, y)}^2 = \frac{1}{K} \sigma_{\eta(x, y)}^2$$

- If K is sufficiently large, then the average image will be very close to the true, original image, and the variance (noise) will be small.

Example

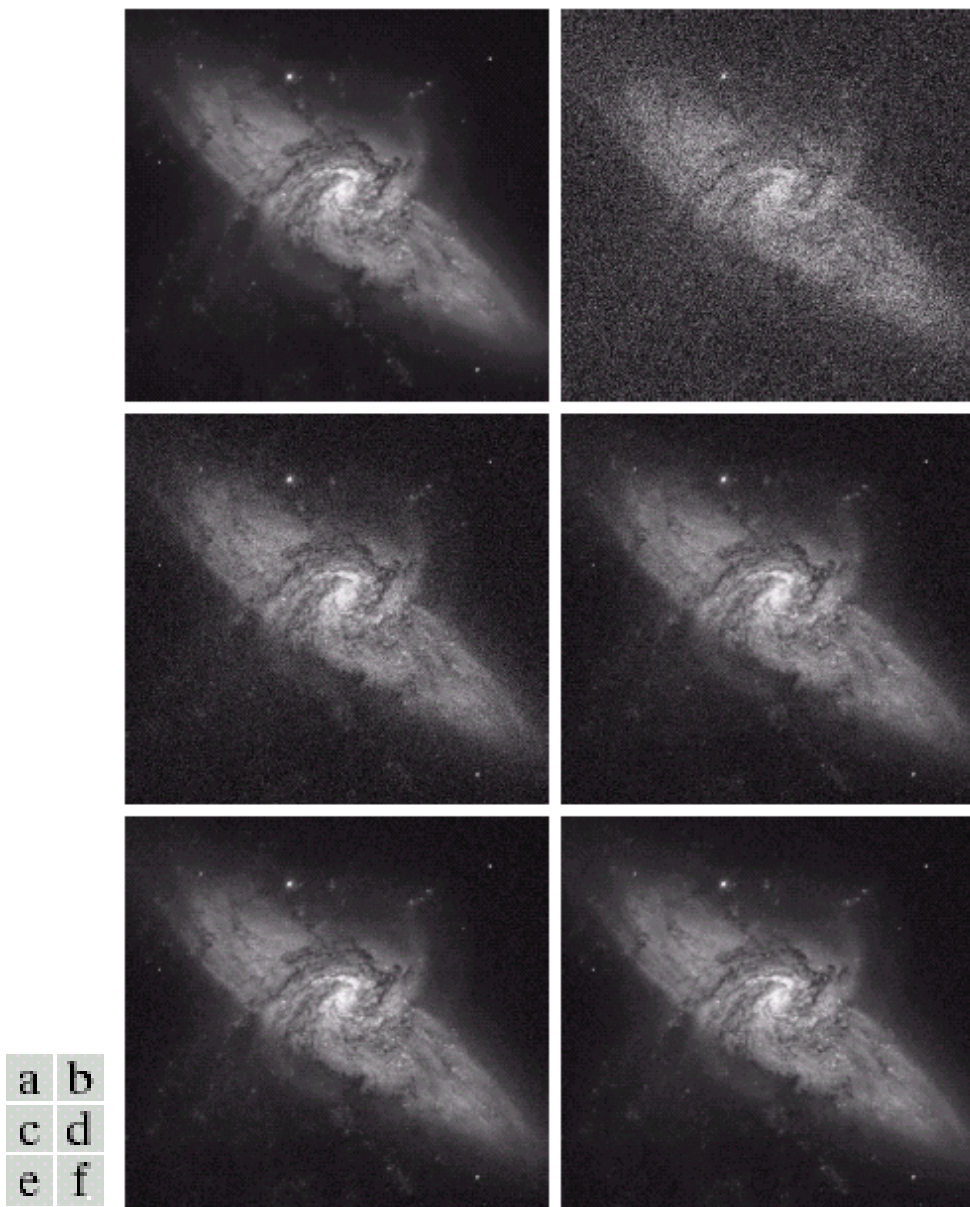


FIGURE 3.30 (a) Image of Galaxy Pair NGC 3314. (b) Image corrupted by additive Gaussian noise with zero mean and a standard deviation of 64 gray levels. (c)–(f) Results of averaging $K = 8, 16, 64$, and 128 noisy images. (Original image courtesy of NASA.)

Spatial Filtering using a Mask/Filter

- Neighborhood operators

z1	z2	z3
z4	z5	z6
z7	z8	z9

mask/filter

w1	w2	w3
w4	w5	w6
w7	w8	w9

Response of a linear mask/filter, R ,

$$R = (w_1z_1 + w_2z_2 + w_3z_3 \dots + w_9z_9) = \sum_{i=1}^9 w_i z_i$$

$f(x,y)$ is centered around "z5"

so $g(x,y) = R = \text{filter} * f(x,y)$, $*$ =convolution symbol

Response of a *linear mask/filter*

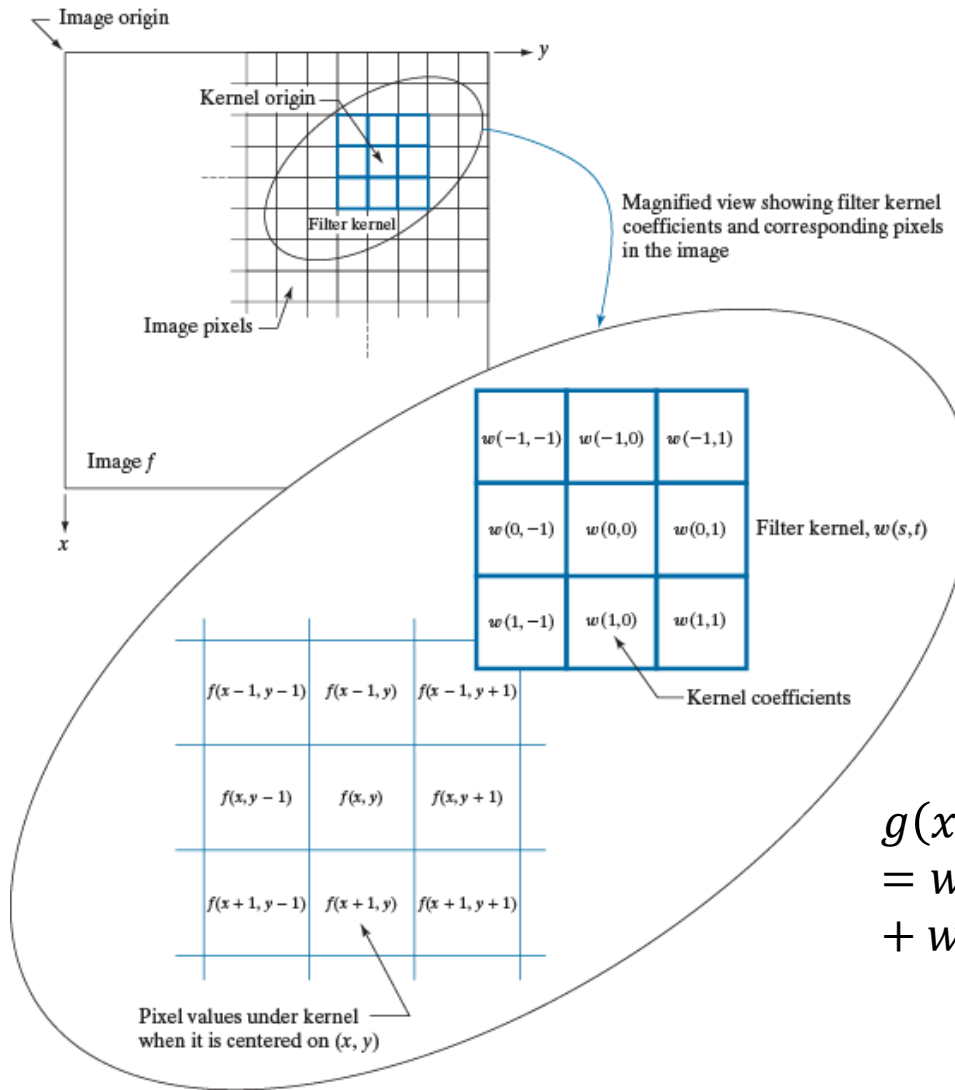
$$R = w_1z_1 + w_2z_2 + w_3z_3 \dots + w_9z_9$$

where w_i are mask coefficients (weights) and z_i are pixel intensities.

Response of a linear mask/filter

FIGURE 3.34

The mechanics of linear spatial filtering using a 3×3 kernel. The pixels are shown as squares to simplify the graphics. Note that the origin of the image is at the top left, but the origin of the kernel is at its center. Placing the origin at the center of spatially symmetric kernels simplifies writing expressions for linear filtering.

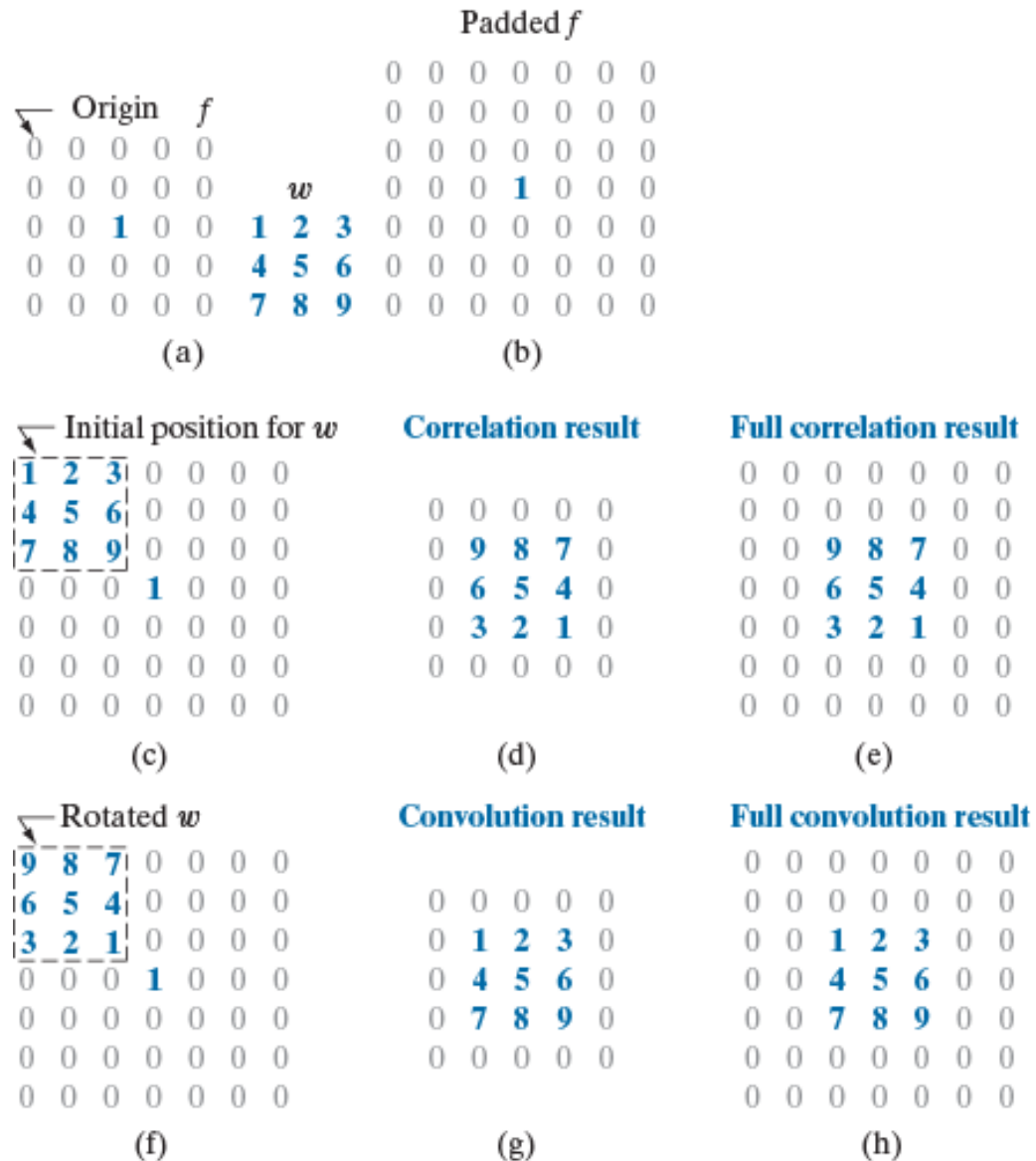


$$g(x, y) = w(-1, -1)f(x-1, y-1) + \dots + w(1, 1)f(x+1, y+1)$$

Response of a linear mask/filter

FIGURE 3.36

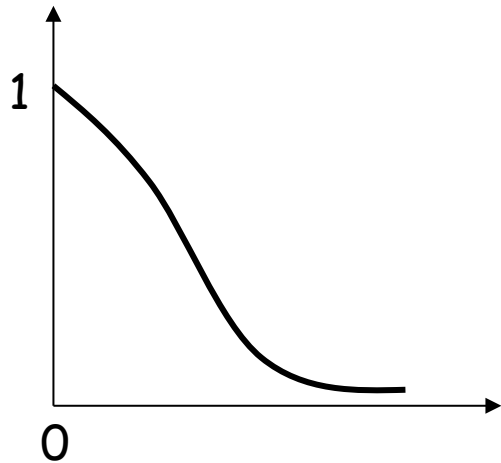
Correlation (middle row) and convolution (last row) of a 2-D kernel with an image consisting of a discrete unit impulse. The 0's are shown in gray to simplify visual analysis. Note that correlation and convolution are functions of x and y . As these variable change, they *displace* one function with respect to the other. See the discussion of Eqs. (3-45) and (3-46) regarding full correlation and convolution.



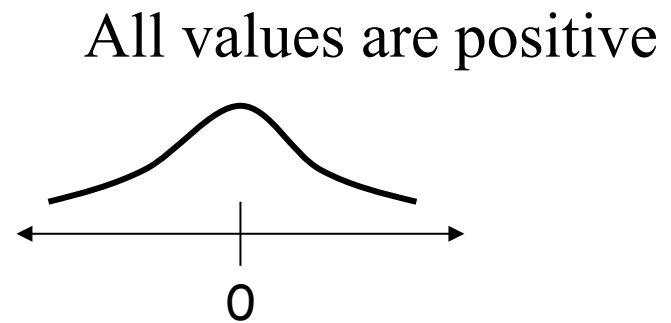
Smoothing Filters

- Smoothing Filters
 - blurring
 - “pre-processing”
 - removal of small details before object extraction
 - noise reduction
 - removal of noise in an image
- Often called “Low-Pass” Filters
 - Filter lets low-frequencies pass
 - Stops high-frequencies

Low-pass spatial filtering



Frequency Domain



Filter center

Spatial Domain

Low-pass spatial filtering

w1	w2	w3
w4	w5	w6
w7	w8	w9

- Only requirement for a low-pass filter is that w_i be positive
- Note that the result can be larger than the valid output range(L-1)
- Can pre-scale the filter

$$\text{scale_factor} = (\sum w_i)^{-1}$$

Low-pass spatial filter

$$\frac{1}{9} * \begin{array}{|c|c|c|} \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline \end{array}$$

Average Filter
(Box Filter)

Extends to larger filters

$$\frac{1}{9} \times \begin{array}{|c|c|c|} \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline \end{array}$$

1/81 *

[illegible]

Example



original



$n=5$ ($n \times n$ mask)



$n=15$ ($n \times n$ mask)



$n=25$ ($n \times n$ mask)

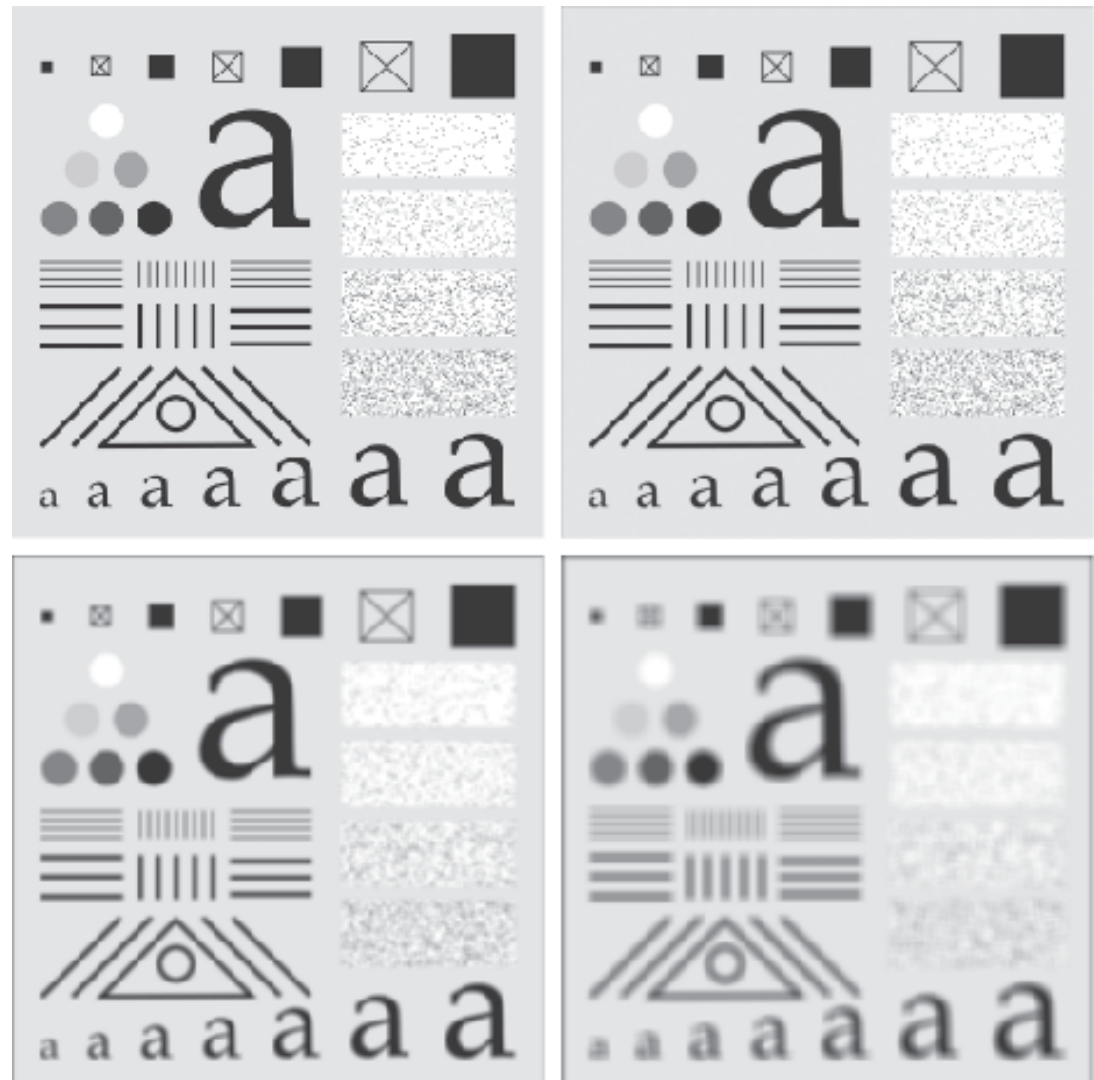
Example

a b
c d

FIGURE 3.39

(a) Test pattern of size 1024×1024 pixels.

(b)-(d) Results of lowpass filtering with box kernels of sizes 3×3 , 11×11 , and 21×21 , respectively.

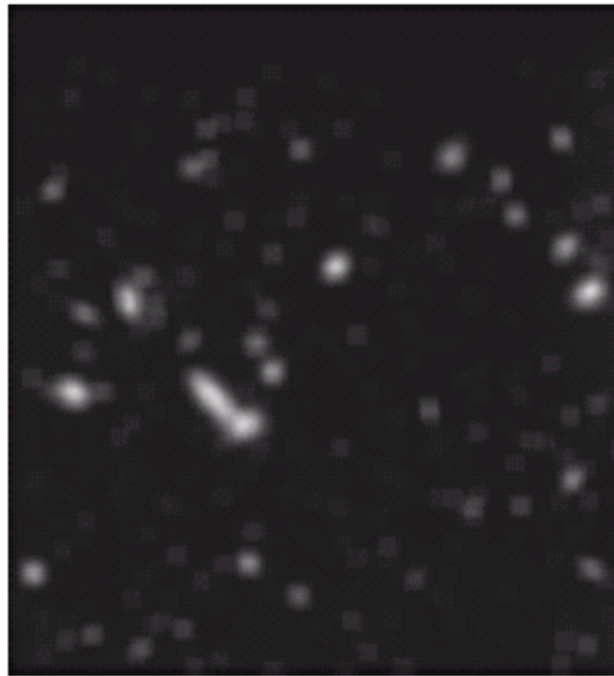
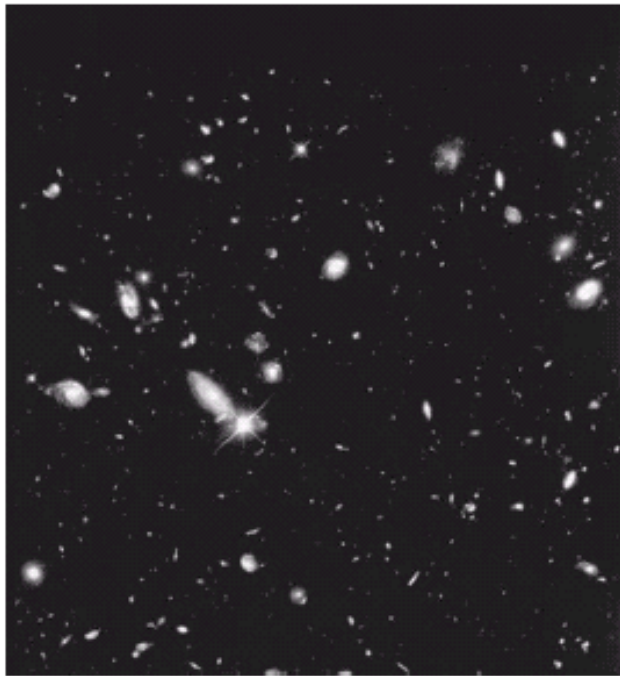


Example

Original image

Smooth image
15x15 averaging filter

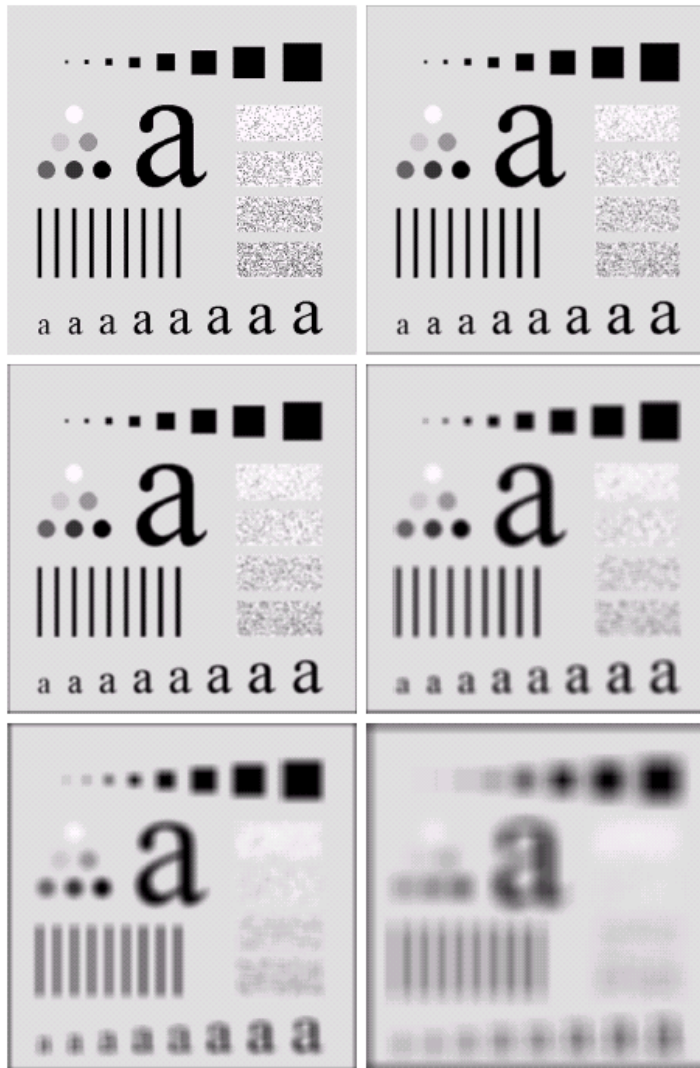
thresholded image



a b c

FIGURE 3.36 (a) Image from the Hubble Space Telescope. (b) Image processed by a 15×15 averaging mask. (c) Result of thresholding (b). (Original image courtesy of NASA.)

Example



a b
c d
e f

FIGURE 3.35 (a) Original image, of size 500×500 pixels. (b)–(f) Results of smoothing with square averaging filter masks of sizes $n = 3, 5, 9, 15$, and 35 , respectively. The black squares at the top are of sizes $3, 5, 9, 15, 25, 35, 45$, and 55 pixels, respectively; their borders are 25 pixels apart. The letters at the bottom range in size from 10 to 24 points, in increments of 2 points; the large letter at the top is 60 points. The vertical bars are 5 pixels wide and 100 pixels high; their separation is 20 pixels. The diameter of the circles is 25 pixels, and their borders are 15 pixels apart; their gray levels range from 0% to 100% black in increments of 20% . The background of the image is 10% black. The noisy rectangles are of size 50×120 pixels.

Other arrangements

1	1	1
1	2	1
1	1	1

1	1	1	1	1
1	2	3	2	1
1	3	4	3	1
1	2	3	2	1
1	1	1	1	1

Remember that the resulting gray-levels may be out of the range of the original image.

Gaussian Smoothing using the Gaussian Filter

The Gaussian distribution in 1-D has the form:

$$G(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{x^2}{2\sigma^2}}$$

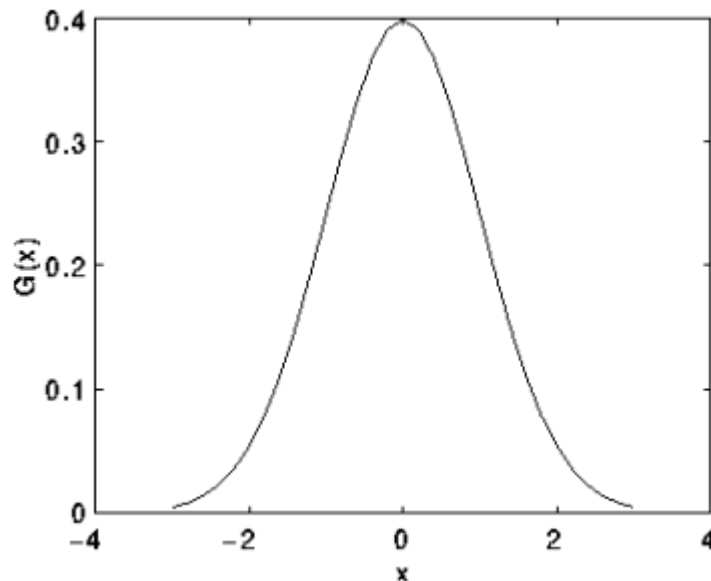


Figure 1 1-D Gaussian distribution with mean 0 and $\sigma=1$

In 2-D, an isotropic (*i.e.* circularly symmetric) Gaussian has the form:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

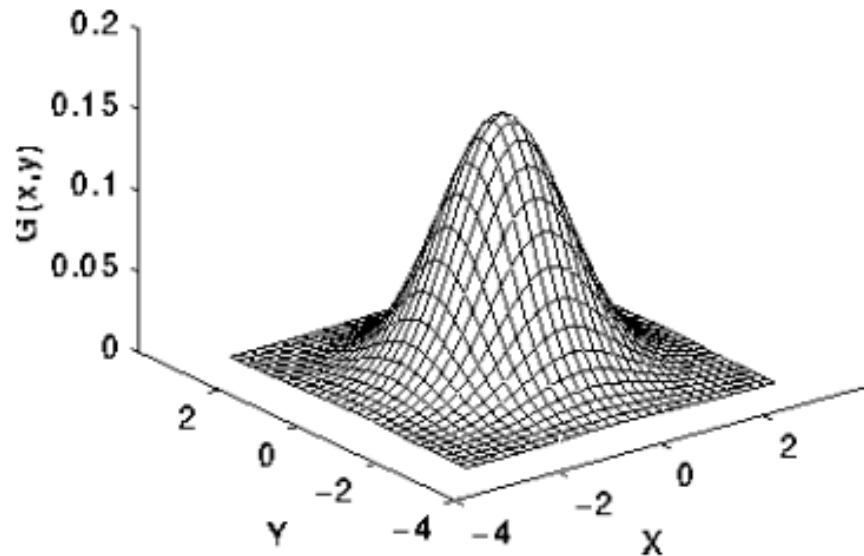


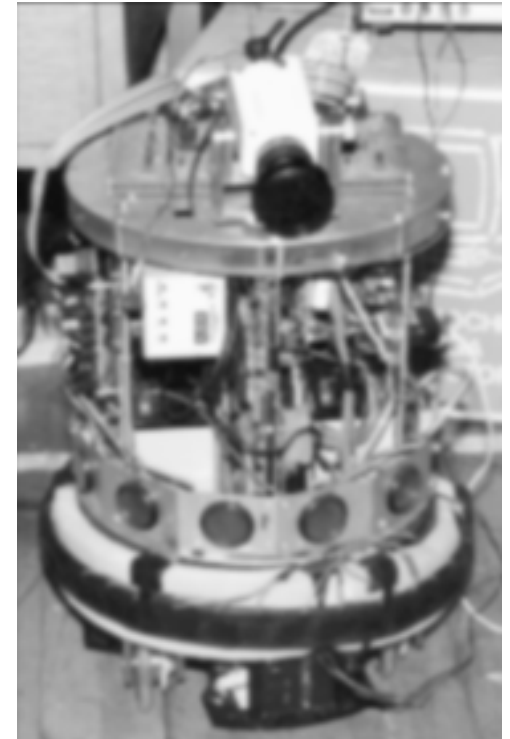
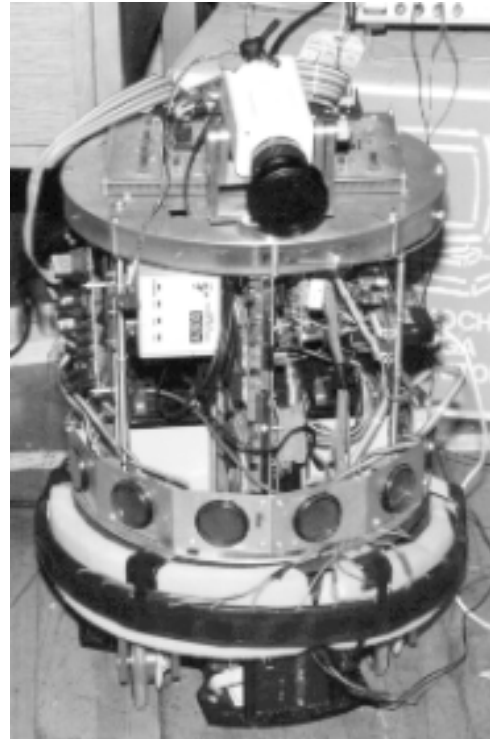
Figure 2 2-D Gaussian distribution with mean (0,0) and $\sigma=1$

Gaussian Smoothing using the Gaussian Filter

$$\frac{1}{273}$$

1	4	7	4	1
4	16	26	16	4
7	26	41	26	7
4	16	26	16	4
1	4	7	4	1

Figure 3 Discrete approximation to Gaussian function with $\sigma=1.0$



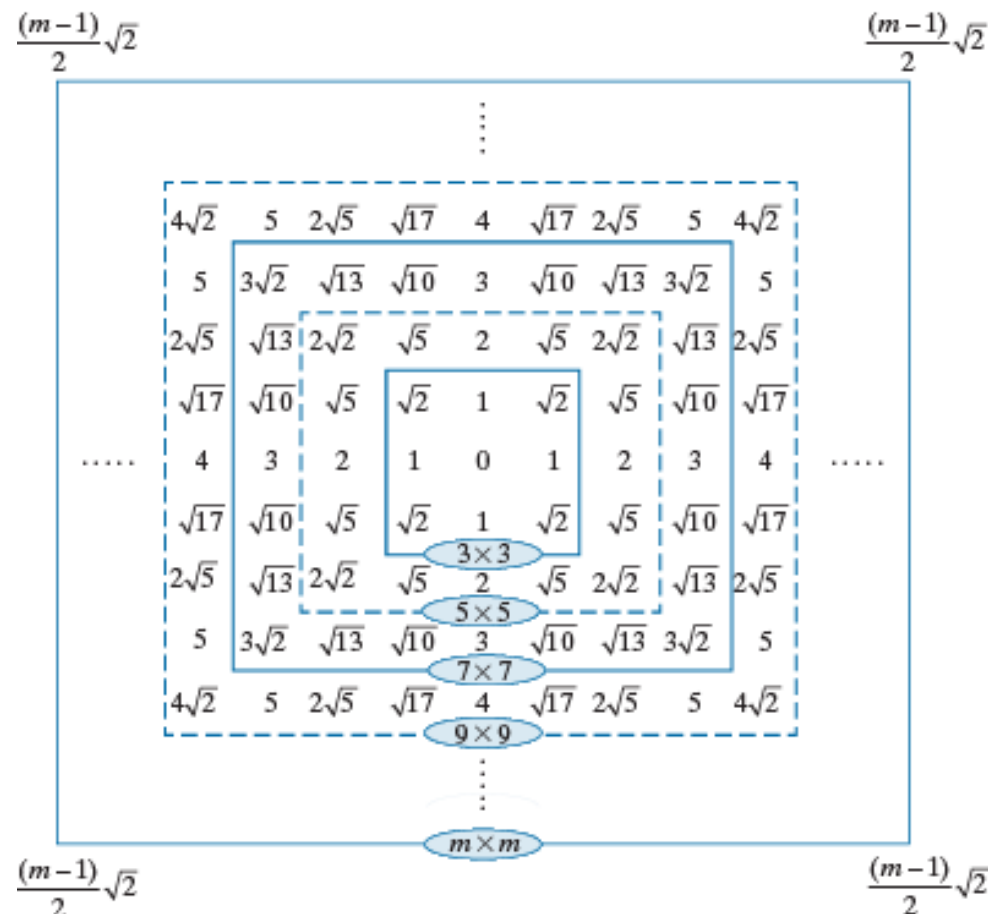
the effect of filtering with a Gaussian of $\sigma = 1.0$ (and kernel size 5×5)

Gaussian Filter

$$w(s, t) = Ke^{-\frac{s^2+t^2}{2\sigma^2}}$$

FIGURE 3.40

Distances from the center for various sizes of square kernels.



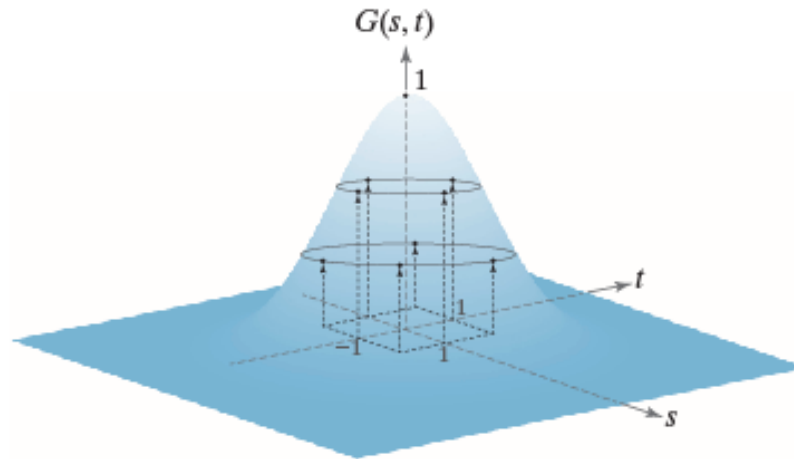
Gaussian Filter

$$w(s, t) = K e^{-\frac{s^2 + t^2}{2\sigma^2}}$$

a b

FIGURE 3.41

(a) Sampling a Gaussian function to obtain a discrete Gaussian kernel. The values shown are for $K = 1$ and $\sigma = 1$. (b) Resulting 3×3 kernel [this is the same as Fig. 3.37(b)].



$\frac{1}{4.8976} \times$

0.3679	0.6065	0.3679
0.6065	1.0000	0.6065
0.3679	0.6065	0.3679

Gaussian Filter



a b c

FIGURE 3.42 (a) A test pattern of size 1024×1024 . (b) Result of lowpass filtering the pattern with a Gaussian kernel of size 21×21 , with standard deviations $\sigma = 3.5$. (c) Result of using a kernel of size 43×43 , with $\sigma = 7$. This result is comparable to Fig. 3.39(d). We used $K = 1$ in all cases.

Linear vs. Non-linear

- Linear Filters
 - Linear operation (output can be expressed as the convolution of the input image with a filter)
 - Have a corresponding frequency domain filter
- Non-linear Filters
 - Examine neighbors using various orderings
 - Often use Order Statistics

Median Filter

- Very popular non-linear filter
- Find the median of the window
- Preserves edges
- Removes impulse noise, avoids excessive smoothing

2	3	8
3	4	10
4	2	9

pixel values about (x,y)
window 3x3

neighbor sort = {2,2,3,3,4,4,8,9,10}



$f(x,y) = \text{median}$

Examples

Original



(a)

Noisy Image



(b)

Low-Pass Filter



(c)

Median Filter



(d)

Examples

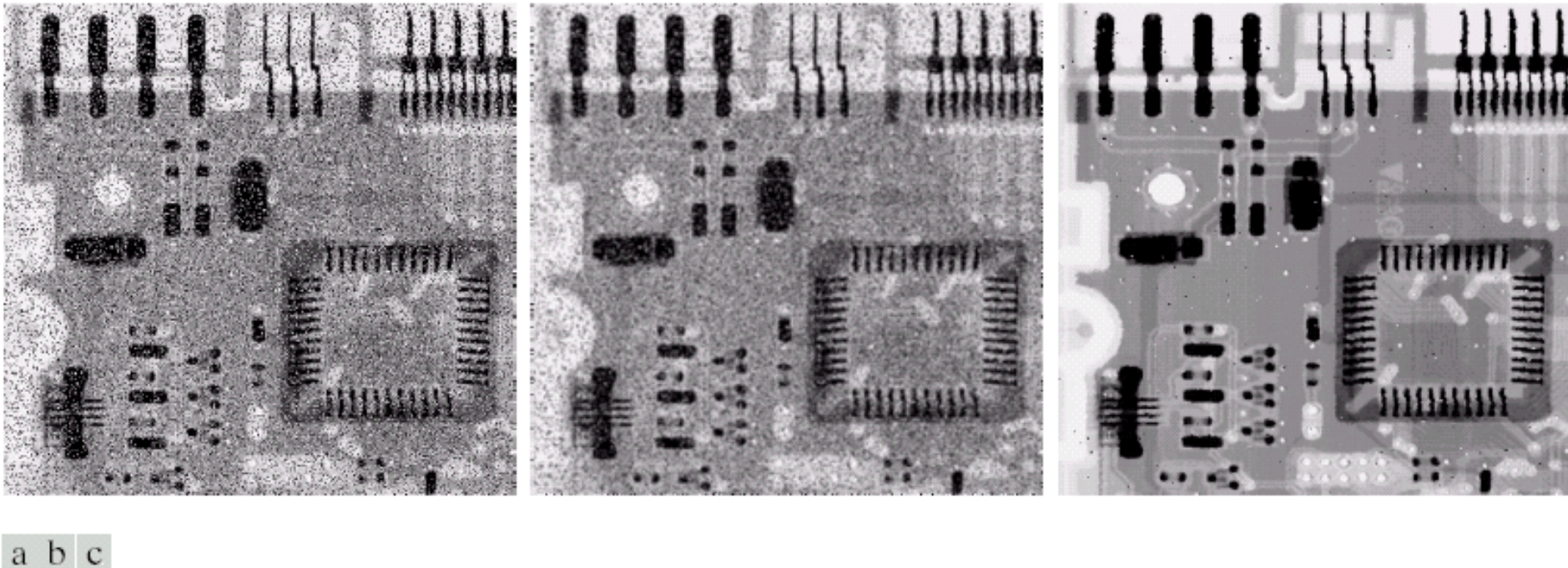


FIGURE 3.37 (a) X-ray image of circuit board corrupted by salt-and-pepper noise. (b) Noise reduction with a 3×3 averaging mask. (c) Noise reduction with a 3×3 median filter. (Original image courtesy of Mr. Joseph E. Pascente, Lixi, Inc.)

Median Filter

- Often referred to as “de-speckle operation”
- It converges
 - That is, if you perform it over and over for many times, eventually the image will not change.

Median Filter Example

- A worked 2D example can be found at https://en.wikipedia.org/wiki/Median_filter



original image



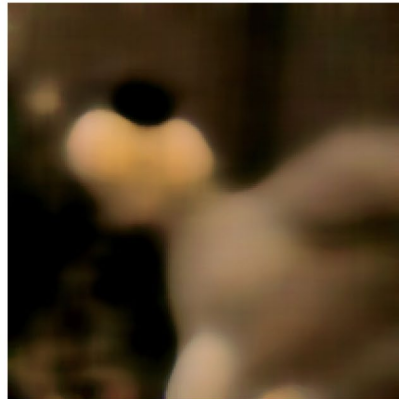
1px median filter



Use of a median filter to improve an image severely corrupted by defective pixels



3px median filter



10px median filter

Example of 3 median filters of varying radii applied to the same noisy photograph.

https://en.wikipedia.org/wiki/Median_filter

Iterative Smoothing

- Local Averaging
 - Converges to an image with constant intensity
- Median Filter
 - Converges to an image invariant to the filter

Sharpening

- Objectives of sharpening
 - Highlight fine details in an image
 - Enhance details that have been blurred
 - We can think of this as high-pass filtering
 - Letting high frequencies pass
 - Removing low frequencies

First Order Derivatives

- Gradient
- Function of 2 variables x, y

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$



First Order Derivatives

- For each (x,y) you are storing two values:

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

- Often have two images to represent this
- X-Gradient and Y-Gradient
 - Can be computed independently

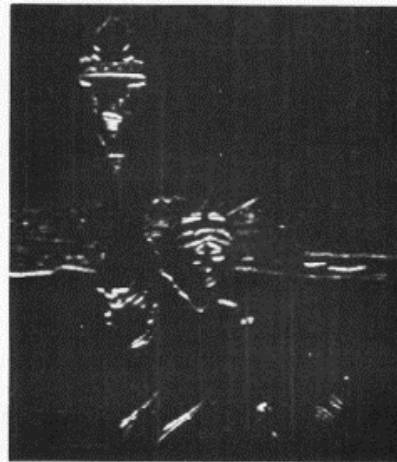
First Order Derivatives

- X-Gradient and Y-Gradient

Original

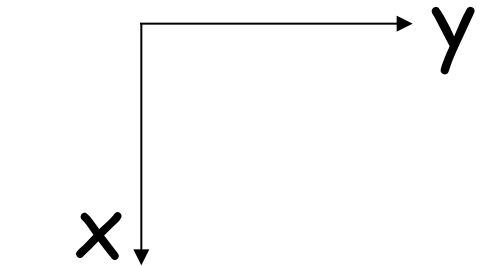


(a)



(b)

$|\partial f / \partial y|$



$|\partial f / \partial x|$

$\text{mag}(\nabla f)$

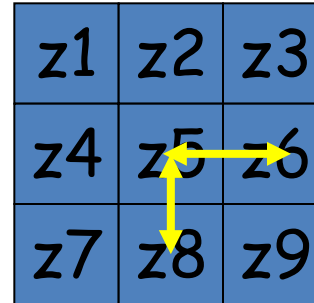
Gradient

- Gradient Magnitude

$$\text{mag}(\nabla f) = \left[\left(\frac{\partial f}{\partial x} \right)^2 + \left(\frac{\partial f}{\partial y} \right)^2 \right]^{1/2}$$

Basic Derivative

- Consider the pixels

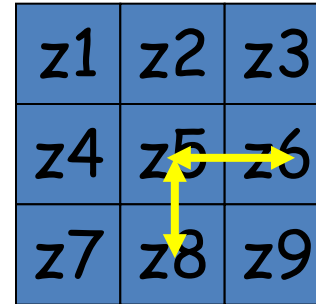


- $\text{mag}(\nabla f)$ at z_5 can be computed

$$\text{mag}(\nabla f) \approx [(z_5 - z_6)^2 + (z_5 - z_8)^2]^{1/2}$$

Basic Derivative

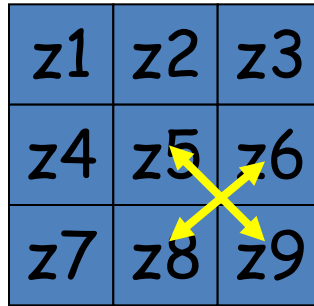
- Consider the pixels



- $\text{mag}(\nabla f)$ at z_5 can be computed quicker

$$\text{mag}(\nabla f) \approx |z_5 - z_6| + |z_5 - z_8|$$

Basic Derivative



- $\text{mag}(\nabla f)$ sometimes is computed using the “cross” difference

$$\text{mag}(\nabla f) \approx |z_6 - z_8| + |z_5 - z_9|$$

First Order Derivatives

z1	z2	z3
z4	z5	z6
z7	z8	z9

$$\text{mag}(\nabla f) \approx |(z_7 + z_8 + z_9) - (z_1 + z_2 + z_3)| + |(z_3 + z_6 + z_9) - (z_1 + z_4 + z_7)|$$

- Difference between first and third rows ($\partial f / \partial x$)
- Difference between first and third columns ($\partial f / \partial y$)

First Order Derivatives

- Prewitt operator

z1	z2	z3
z4	z5	z6
z7	z8	z9

-1	0	1
-1	0	1
-1	0	1

-1	-1	-1
0	0	0
1	1	1

$$\text{mag}(\nabla f) \approx |(z_7 + z_8 + z_9) - (z_1 + z_2 + z_3)| + \\ |(z_3 + z_6 + z_9) - (z_1 + z_4 + z_7)|$$

First Order Derivatives

- Sobel operator

-1	0	1
-2	0	2
-1	0	1

-1	-2	-1
0	0	0
1	2	1

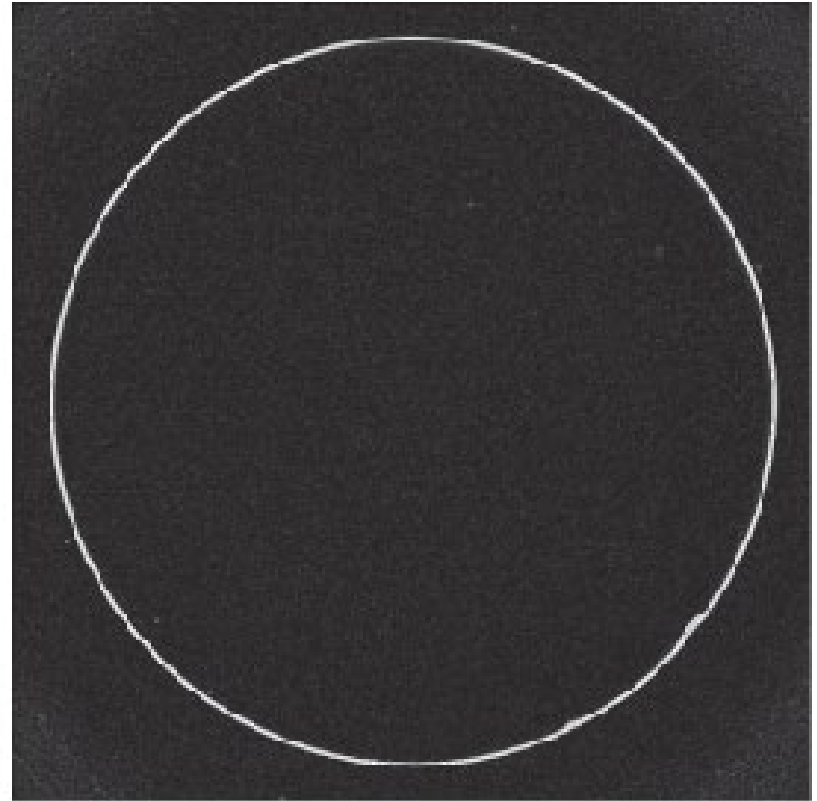
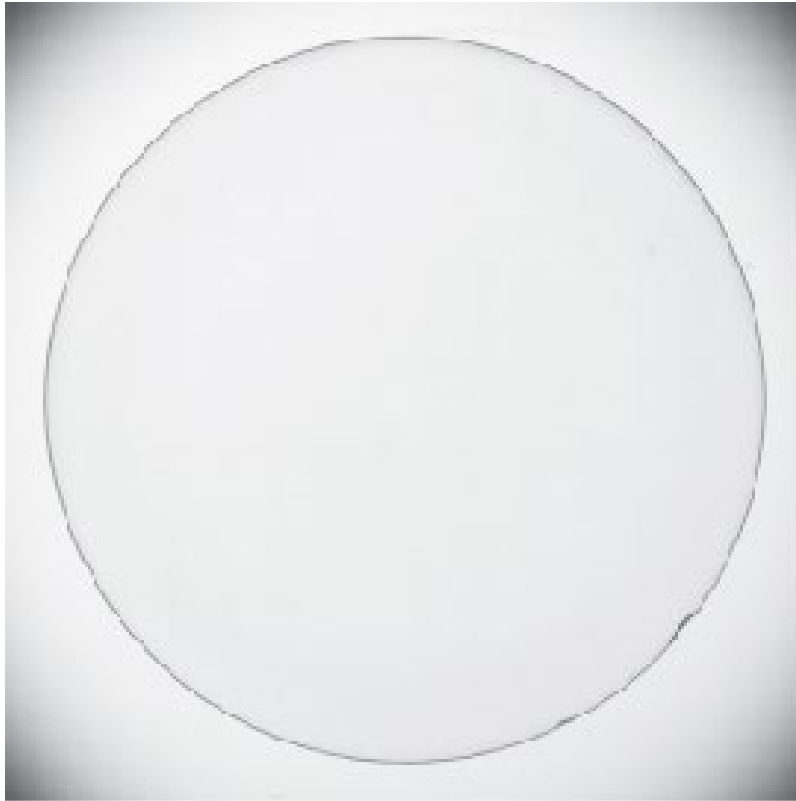
- Weights closer neighbor a little more

Example

a b

FIGURE 3.57

(a) Image of a contact lens (note defects on the boundary at 4 and 5 o'clock).
(b) Sobel gradient.
(Original image courtesy of Perceptics Corporation.)



Unsharp Masking and Highboost Filtering

1. Blur the original image $f(x, y)$ to generate $\bar{f}(x, y)$
2. Subtract the blurred image $\bar{f}(x, y)$ from the original image $f(x, y)$ (unsharp masking)
3. Add the mask to the original image (highboost filtering) to generate $g(x, y)$

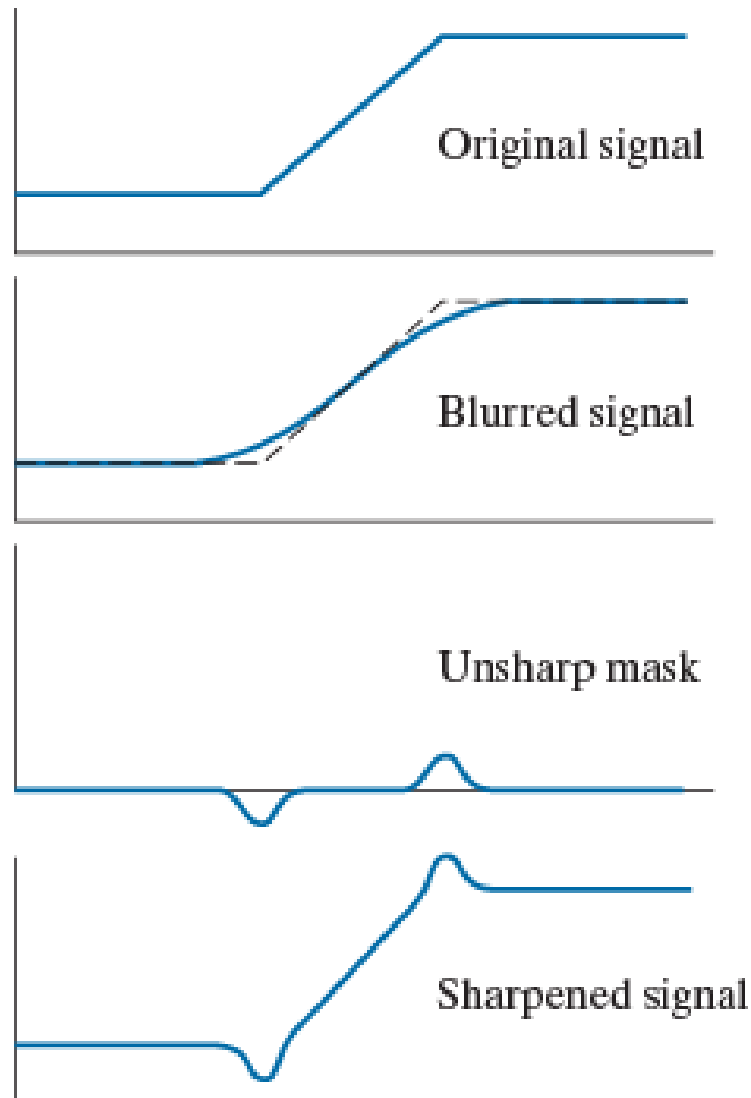
$$g(x, y) = f(x, y) + k(f(x, y) - \bar{f}(x, y))$$

Unsharp Masking and Highboost Filtering

a
b
c
d

FIGURE 3.54

1-D illustration of the mechanics of unsharp masking. (a) Original signal. (b) Blurred signal with original shown dashed for reference. (c) Unsharp mask. (d) Sharpened signal, obtained by adding (c) to (a).



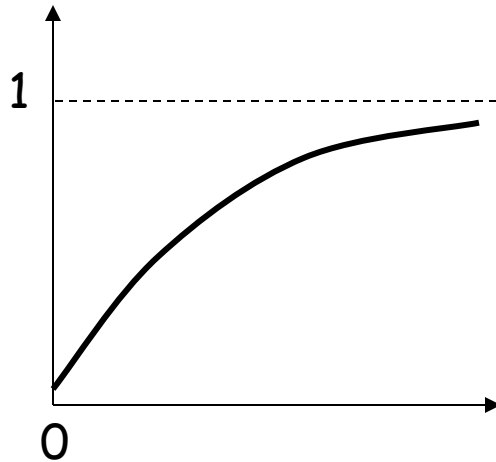
Unsharp Masking and Highboost Filtering



a b c
d e f

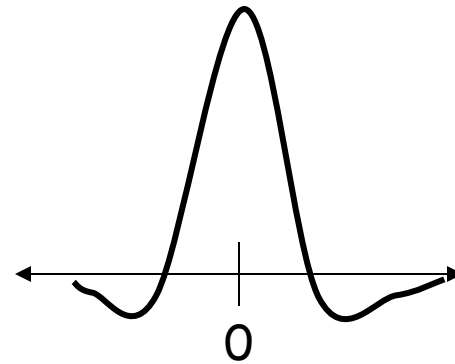
FIGURE 3.55 (a) Unretouched “soft-tone” digital image of size 469×600 pixels. (b) Image blurred using a 31×31 Gaussian lowpass filter with $\sigma = 5$. (c) Mask. (d) Result of unsharp masking using Eq. (3-65) with $k = 1$. (e) and (f) Results of highboost filtering with $k = 2$ and $k = 3$, respectively.

Sharpening



Frequency Domain

Values can be either positive or negative



Filter center

Spatial Domain

Sharpening spatial filter

- The results may be negative.
- You'll need to scale and/or clip so that the gray levels of the result span $[0, L-1]$.

Sharpening spatial filter

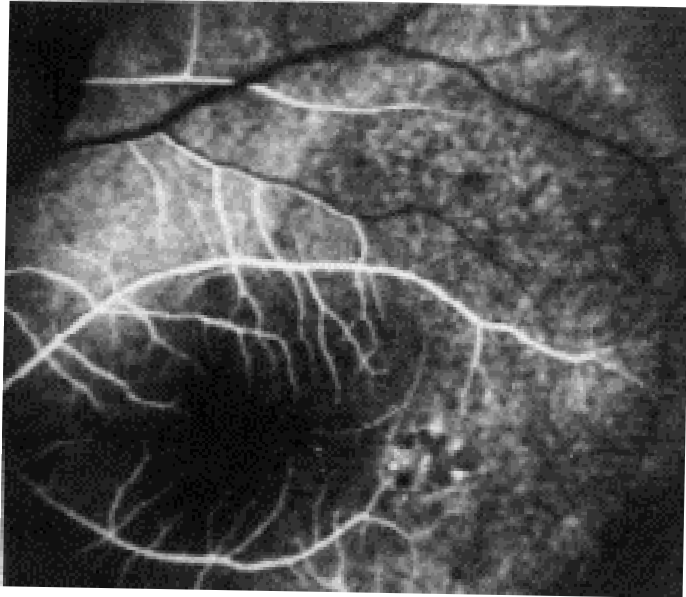
- Positive coefficients near its center
- Negative coefficients near the outer periphery

$1/9 *$

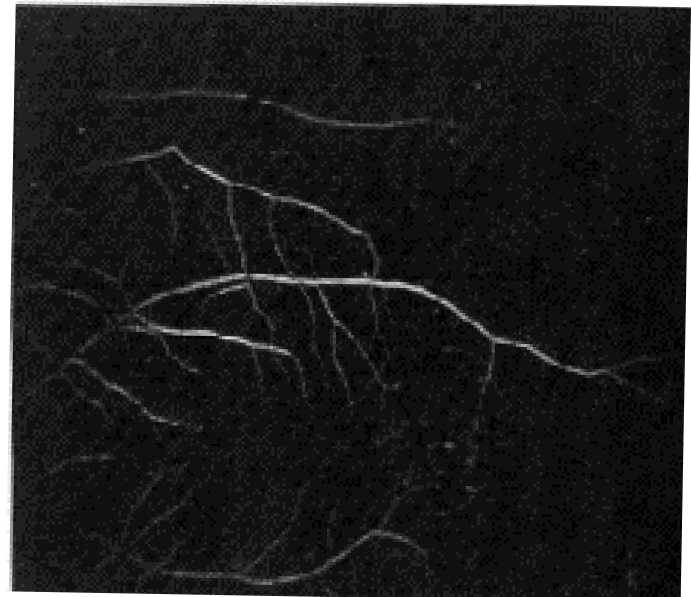
-1	-1	-1
-1	8	-1
-1	-1	-1

- Note, the sum of the coefficients is zero
- Thus, when the mask is applied over an area of constant intensity, the result is zero
- Output = 0 if the intensity values are constant.

Example



Original



High-Pass

Second Order Derivatives

- The Laplacian operator

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

- Central difference approximation along the x-direction and the y-direction

$$\begin{aligned} \nabla^2 f &= f(x+1, y) + f(x-1, y) + f(x, y+1) \\ &\quad + f(x, y-1) - 4f(x, y) \end{aligned}$$

- How to derive?

Second Order Derivatives

0	1	0	1	1	1	0	-1	0	-1	-1	-1
1	-4	1	1	-8	1	-1	4	-1	-1	8	-1
0	1	0	1	1	1	0	-1	0	-1	-1	-1

a b c d

FIGURE 3.51 (a) Laplacian kernel used to implement Eq. (3-62). (b) Kernel used to implement an extension of this equation that includes the diagonal terms. (c) and (d) Two other Laplacian kernels.

Second Order Derivatives

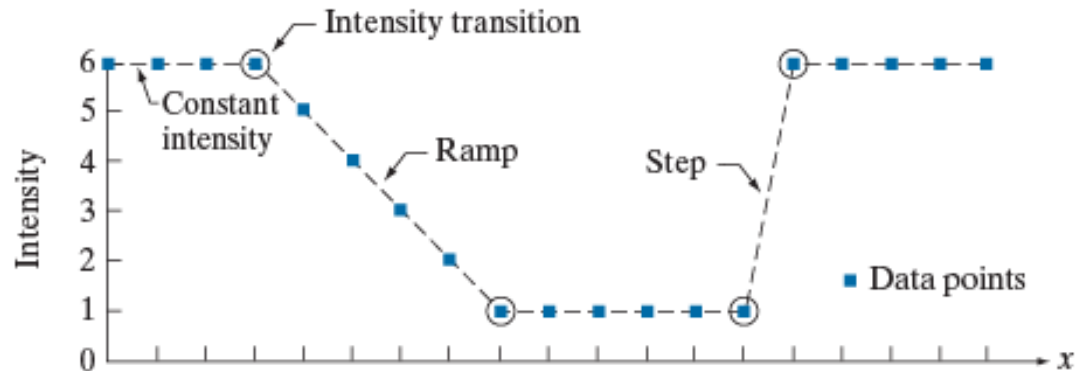
a
b
c

FIGURE 3.50

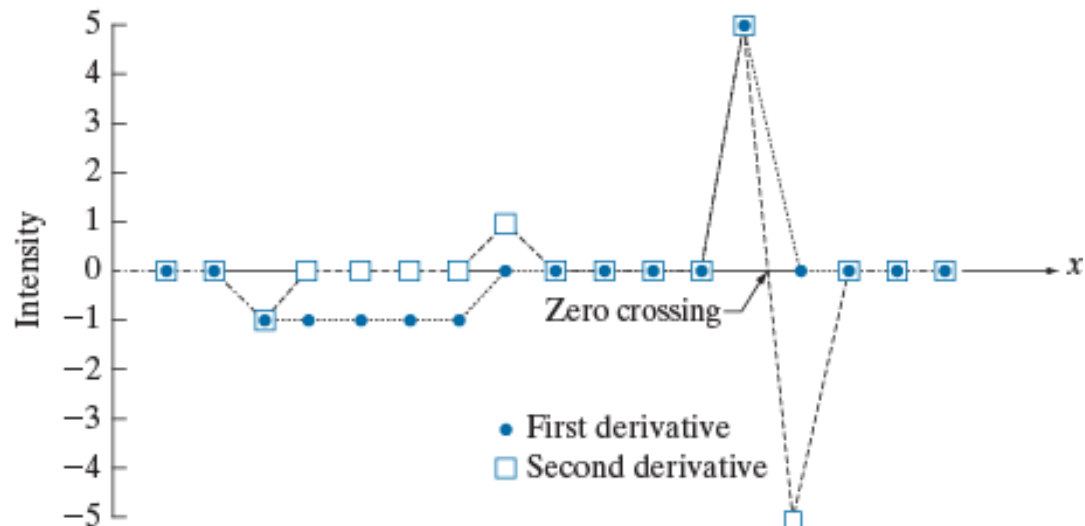
(a) A section of a horizontal scan line from an image, showing ramp and step edges, as well as constant segments.

(b) Values of the scan line and its derivatives.

(c) Plot of the derivatives, showing a zero crossing. In (a) and (c) points were joined by dashed lines as a visual aid.



Values of scan line	6	6	6	6	5	4	3	2	1	1	1	1	1	1	6	6	6	6	6	x
1st derivative	0	0	-1	-1	-1	-1	-1	0	0	0	0	0	0	5	0	0	0	0	0	
2nd derivative	0	0	-1	0	0	0	0	1	0	0	0	0	0	5	-5	0	0	0	0	



Laplacian for sharpening

$$g(x, y) = f(x, y) + c[\nabla^2 f(x, y)]$$

$$c = -1$$

Why is c negative?

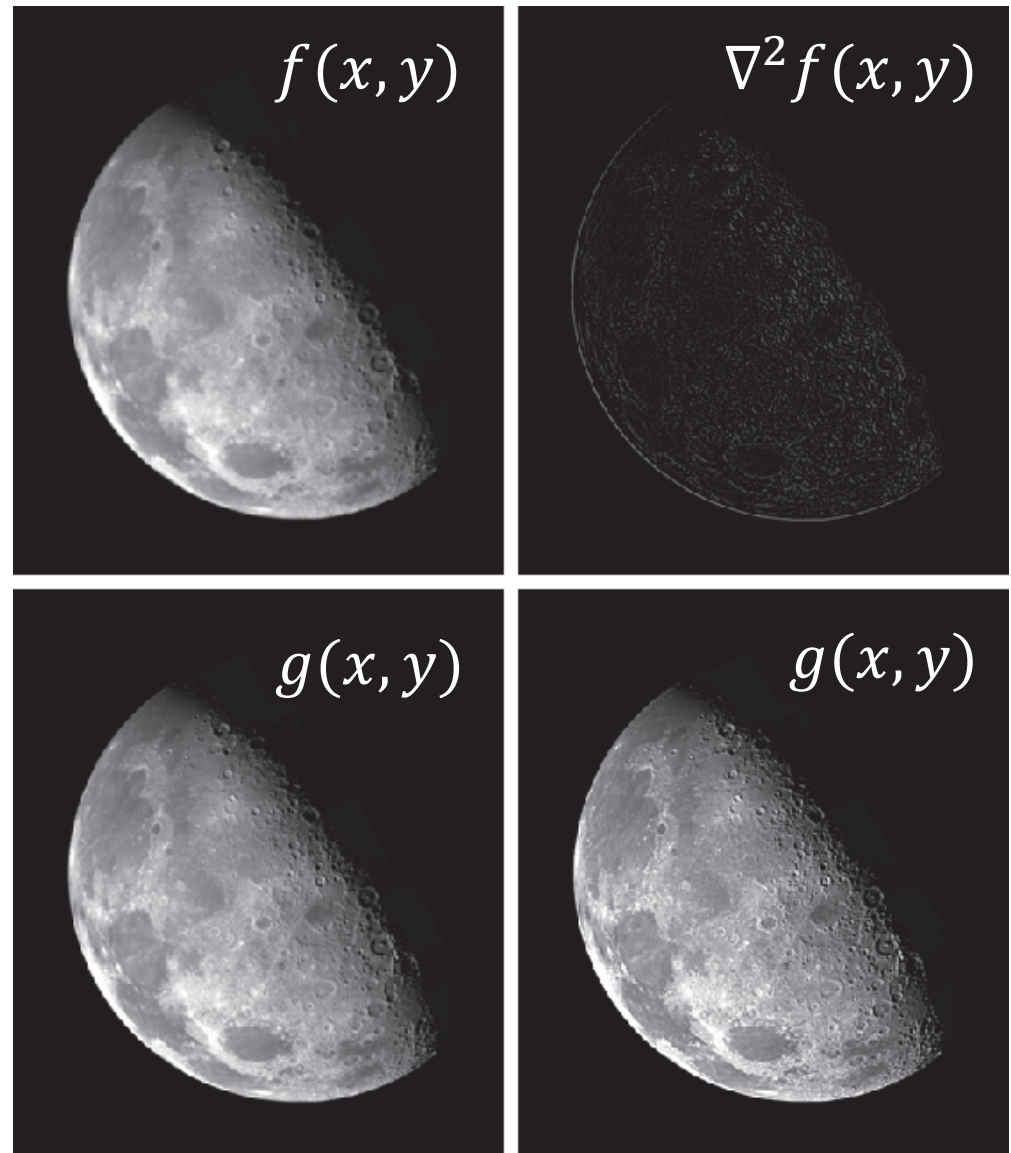


FIGURE 3.52
(a) Blurred image of the North Pole of the moon.
(b) Laplacian image obtained using the kernel in Fig. 3.51(a).
(c) Image sharpened using Eq. (3-63) with $c = -1$.
(d) Image sharpened using the same procedure, but with the kernel in Fig. 3.51(b). (Original image courtesy of NASA.)

Laplacian for sharpening

Original



Sharpened



$$c = -0.7$$

Original



Sharpened



Bilateral Filtering

A Gentle Introduction to Bilateral Filtering and its Applications

Sylvain Paris¹ Pierre Kornprobst² Jack Tumblin³ Frédo Durand¹

¹ Computer Science and Artificial Intelligence Laboratory
Massachusetts Institute of Technology

² Odyssee Lab – INRIA, ENS , ENPC (France)

³ Department of Computer Science, Northwestern University

This course reviews the wealth of work related to bilateral filtering. The bilateral filter is ubiquitous in computational photography applications. It is increasingly common in computer graphics research papers but no single reference summarizes its properties and applications. This course provides a graphical, intuitive introduction to bilateral filtering, and a practical guide for image editing, tone-mapping, video processing and more.

Bilateral Filtering

Gaussian Blur filter

$$GB[I]_p = \sum_{q \in S} G_\sigma(\|p - q\|) I_q$$

$$G_\sigma(x) = \frac{1}{2\pi\sigma^2} e^{\left(-\frac{x^2}{2\sigma^2}\right)}$$

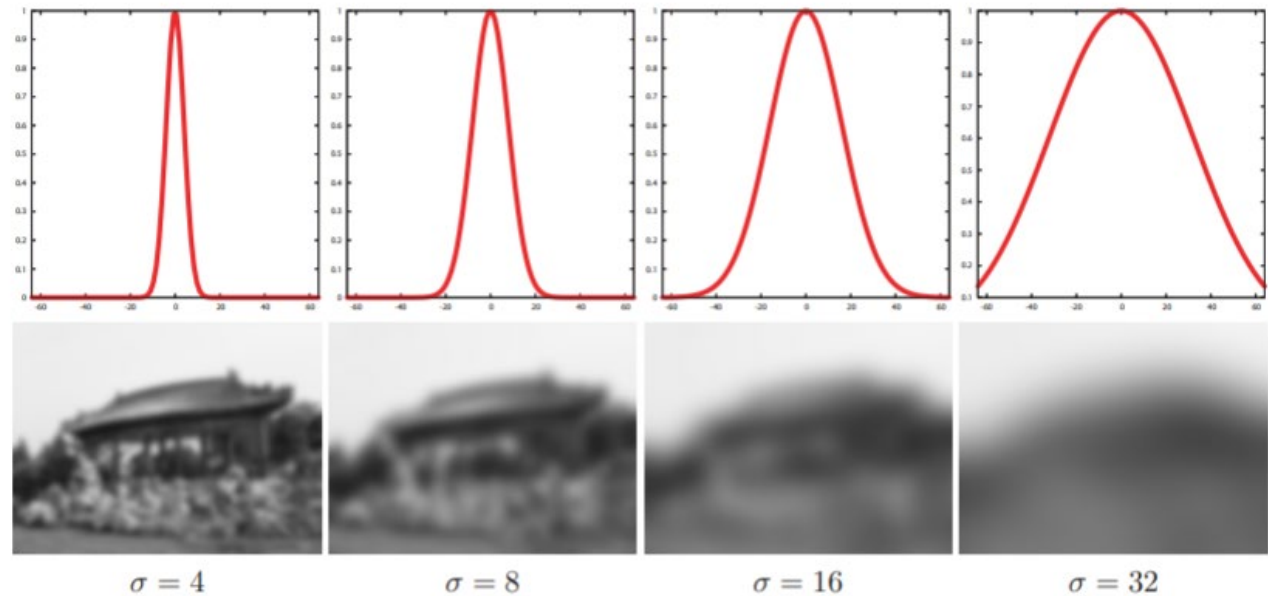


Figure 3: Example of Gaussian linear filtering with different σ . Top row show the profile of a 1D Gaussian kernel and bottom row the result obtained by the corresponding 2D Gaussian blur filtering. Edges are lost with high values of σ since more averaging is performed.

Bilateral Filtering

Bilateral filter

$$BF[I]_p = \frac{1}{W_p} \sum_{q \in S} G_\sigma(\|p - q\|) G_{\sigma'}(I_p - I_q) I_q$$

$$G_\sigma(x) = \frac{1}{2\pi\sigma^2} e^{\left(-\frac{x^2}{2\sigma^2}\right)}$$

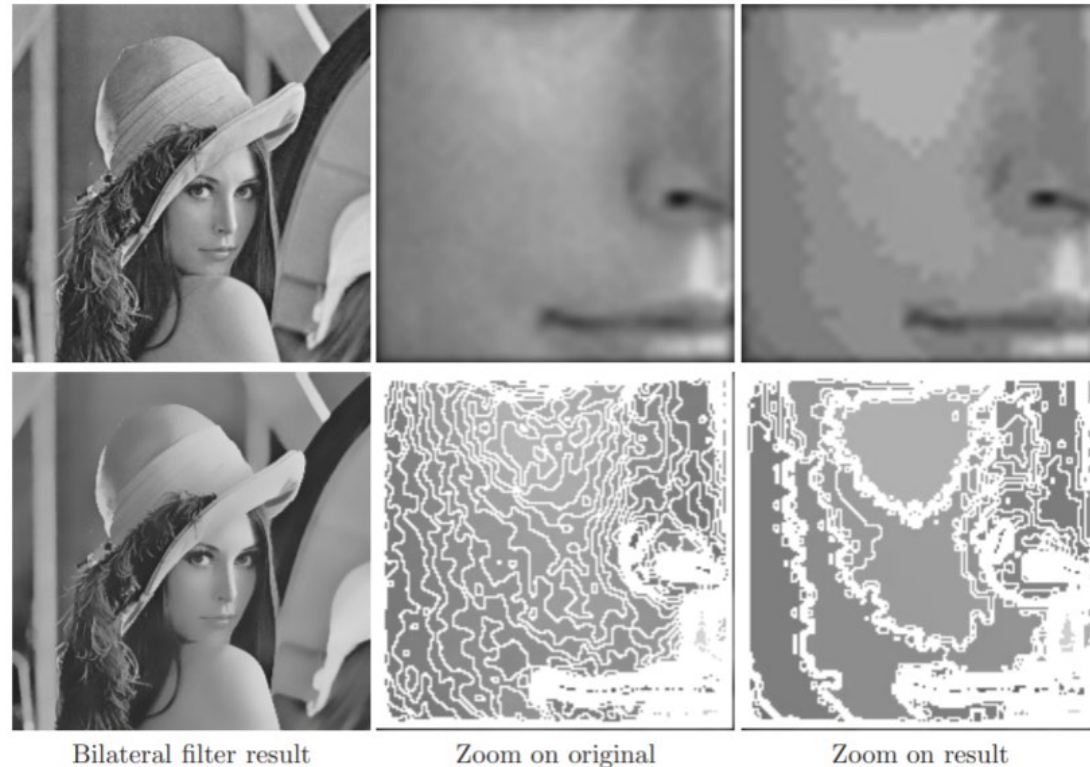


Figure 4: Example of result obtained with the Bilateral filter. First columns shown the original image (top) and the smoothed image (bottom). Second column is a zoom on one part of the image (top) with corresponding level lines below. Third column is the same illustration but for the result. Note that a “staircase effect” is visible, we discuss it in Section 7.5. Zoom images are reproduced from [Buades et al., 2006].

Bilateral Filtering

Bilateral filter

$$BF[I]_p = \frac{1}{W_p} \sum_{q \in S} G_{\sigma}(\|p - q\|) G_{\sigma'}(I_p - I_q) I_q$$

$$G_{\sigma}(x) = \frac{1}{2\pi\sigma^2} e^{\left(-\frac{x^2}{2\sigma^2}\right)}$$

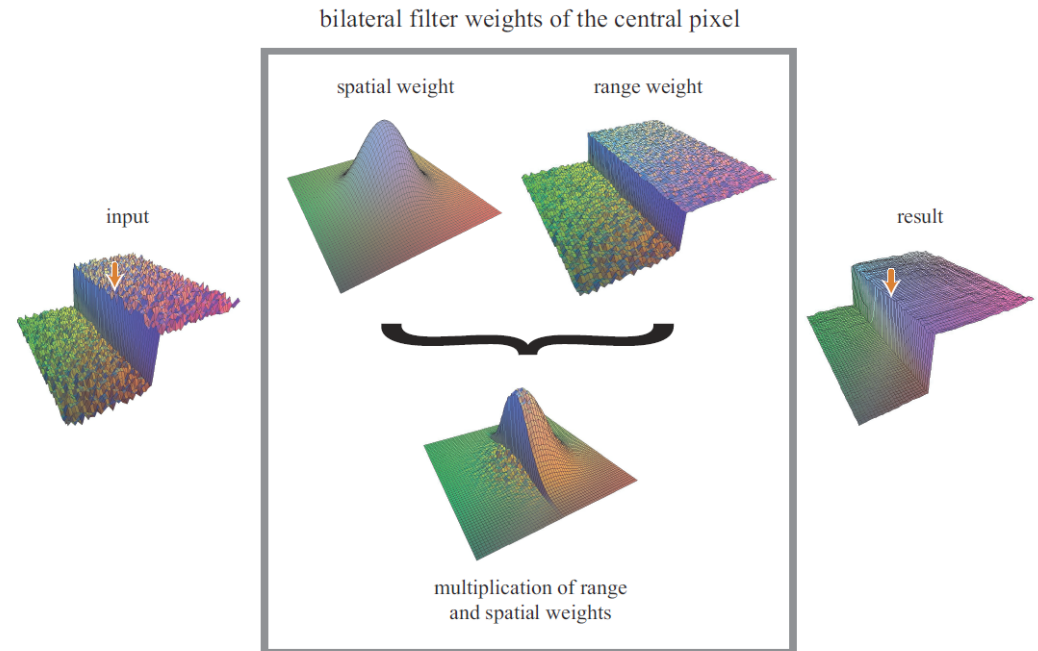


Figure 5: The bilateral filter smooths an input image while preserving its edges. Each pixel is replaced by a weighted average of its neighbors. Each neighbor is weighted by a spatial component that penalizes distant pixels and range component that penalizes pixels with a different intensity. The combination of both components ensures that only nearby similar pixels contribute to the final result. The weights are represented for the central pixel (under the arrow). The figure is reproduced from: Fast bilateral filtering for the display of high-dynamic-range images *Durand and Dorsey* ACM SIGGRAPH conference (c) 2002, Association for Computing Machinery, Inc. Reprinted by permission. <http://doi.acm.org/10.1145/566570.566574>

Bilateral Filtering



Input



Bilateral filter



Residual

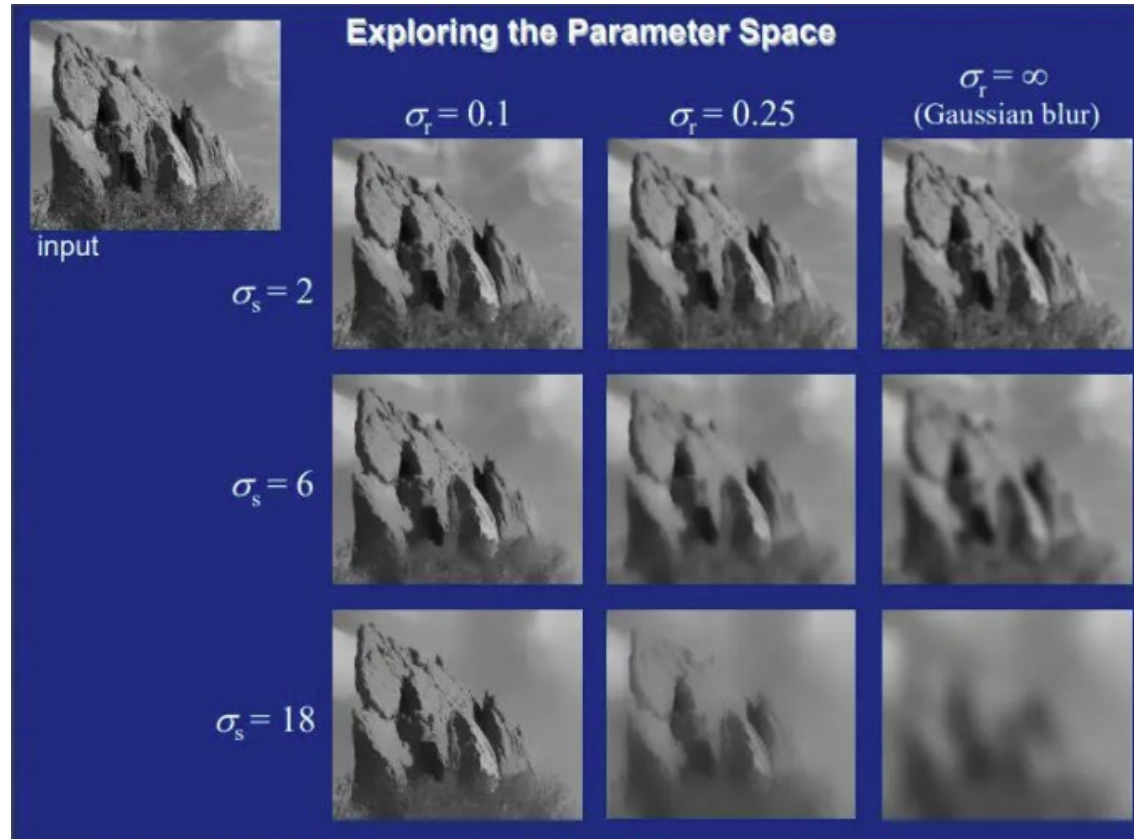
Figure 8: Separation: The residual of the bilateral filter reveals the structure of the input image: Ideally for denoising tasks, the residual should contain only noise. Nonetheless, the structure visible in the residual is faint and bilateral filtering yields acceptable results in numerous denoising scenarios.

Bilateral Filtering

$$BF[I]_{\mathbf{p}} = \frac{1}{W_{\mathbf{p}}} \sum_{\mathbf{q} \in \mathcal{S}} G_{\sigma_s}(\|\mathbf{p} - \mathbf{q}\|) G_{\sigma_r}(I_{\mathbf{p}} - I_{\mathbf{q}}) I_{\mathbf{q}}$$

where $W_{\mathbf{p}}$ is a normalization factor

$$W_{\mathbf{p}} = \sum_{\mathbf{q} \in \mathcal{S}} G_{\sigma_s}(\|\mathbf{p} - \mathbf{q}\|) G_{\sigma_r}(I_{\mathbf{p}} - I_{\mathbf{q}})$$



Bilateral Filtering – Parameters (Source)